

13. JDBC (Java Database Connectivity) (continued)

Stored Procedures (continued):


```

// Calling stored procedure with OUT parameter
try (Connection conn = DriverManager.getConnection(url, username, password);
     CallableStatement cs = conn.prepareCall("{call get_user_count(?)}")) {

    // Register OUT parameter
    cs.registerOutParameter(1, Types.INTEGER);

    // Execute
    cs.execute();

    // Get result
    int count = cs.getInt(1);
    System.out.println("User count: " + count);
}

// Calling stored procedure with IN and OUT parameters
try (Connection conn = DriverManager.getConnection(url, username, password);
     CallableStatement cs = conn.prepareCall("{call calculate_bonus(?, ?)}")) {

    // Set IN parameter
    cs.setInt(1, 1001); // Employee ID

    // Register OUT parameter
    cs.registerOutParameter(2, Types.DOUBLE);

    // Execute
    cs.execute();

    // Get result
    double bonus = cs.getDouble(2);
    System.out.println("Calculated bonus: " + bonus);
}

// Stored procedure returning result set
try (Connection conn = DriverManager.getConnection(url, username, password);
     CallableStatement cs = conn.prepareCall("{call get_users_by_age(?)}")) {

    cs.setInt(1, 30);

    // Execute and process result set
    boolean hasResults = cs.execute();
    if (hasResults) {
        try (ResultSet rs = cs.getResultSet()) {
            while (rs.next()) {
                System.out.println("User: " + rs.getString("name"));
            }
        }
    }
}

```

```
}  
}  
}
```

Metadata:

java



```
try (Connection conn = DriverManager.getConnection(url, username, password)) {  
    // Database metadata  
    DatabaseMetaData dbmd = conn.getMetaData();  
  
    System.out.println("Database: " + dbmd.getDatabaseProductName());  
    System.out.println("Version: " + dbmd.getDatabaseProductVersion());  
  
    // List tables  
    try (ResultSet tables = dbmd.getTables(null, null, "%", new String[]{"TABLE"})) {  
        System.out.println("Tables:");  
        while (tables.next()) {  
            System.out.println("    " + tables.getString("TABLE_NAME"));  
        }  
    }  
  
    // Result set metadata  
    try (Statement stmt = conn.createStatement();  
        ResultSet rs = stmt.executeQuery("SELECT * FROM users")) {  
        ResultSetMetaData rsmd = rs.getMetaData();  
        int columnCount = rsmd.getColumnCount();  
  
        System.out.println("Columns in users table:");  
        for (int i = 1; i <= columnCount; i++) {  
            System.out.printf("    Column %d: %s (%s)%n",  
                              i, rsmd.getColumnName(i), rsmd.getColumnTypeName(i));  
        }  
    }  
}
```

14. Java Memory Management

Memory Areas in JVM:

- **Heap:** Storage for all objects created by the application
 - Young Generation (Eden, Survivor spaces)
 - Old Generation (Tenured)

- **Method Area:** Stores class structures, methods, static variables
- **Stack:** Stores local variables and method call frames (one per thread)
- **PC Register:** Stores current instruction pointer for each thread
- **Native Method Stack:** Used for native methods

Garbage Collection:

java



```
// Making objects eligible for garbage collection
Person person = new Person("John");
person = null; // Reference set to null, object eligible for GC

// Another way - reference goes out of scope
{
    Person tempPerson = new Person("Alice");
    // tempPerson is only available in this block
} // tempPerson goes out of scope, object eligible for GC

// Requesting GC (no guarantee it will run)
System.gc(); // Or Runtime.getRuntime().gc();
```

Memory Management Best Practices:

java



```
// Use try-with-resources for automatic resource closing
try (FileInputStream fis = new FileInputStream("file.txt")) {
    // Use resource
} // File is automatically closed

// Avoid memory leaks in collections
Map<Key, Value> cache = new WeakHashMap<>(); // Keys are weakly referenced

// Explicit nulling of references when done
byte[] largeArray = new byte[1000000];
// Use largeArray
largeArray = null; // Make it eligible for GC when no longer needed

// Using soft references for caches
SoftReference<ExpensiveObject> softRef = new SoftReference<>(new ExpensiveObject());
ExpensiveObject obj = softRef.get(); // May return null if GC reclaimed it
```

15. Java Annotations

Built-in Annotations:

java



```
// For methods overriding superclass methods
@Override
public String toString() {
    return "Overridden toString";
}

// For deprecated API elements
@Deprecated
public void oldMethod() {
    // This method is no longer recommended
}

// For suppressing compiler warnings
@SuppressWarnings("unchecked")
public void uncheckedOperation() {
    List list = new ArrayList();
    list.add("string");
}

// For functional interfaces
@FunctionalInterface
public interface MyFunction {
    void apply();
    // Only one abstract method allowed
}

// For safe varargs
@SafeVarargs
public final <T> List<T> asList(T... elements) {
    List<T> list = new ArrayList<>();
    for (T element : elements) {
        list.add(element);
    }
    return list;
}
```

Custom Annotations:

```
import java.lang.annotation.*;

// Defining custom annotation
@Retention(RetentionPolicy.RUNTIME) // When to discard the annotation
@Target(ElementType.METHOD)        // Where it can be applied
public @interface Test {
    String description() default "No description";
    boolean enabled() default true;
}

// Using custom annotation
class MyTestClass {
    @Test(description = "Test method", enabled = true)
    public void testMethod() {
        // Test code
    }
}

// Processing annotations at runtime
import java.lang.reflect.Method;

public class TestRunner {
    public void runTests(Class<?> testClass) throws Exception {
        Object instance = testClass.getDeclaredConstructor().newInstance();

        for (Method method : testClass.getDeclaredMethods()) {
            Test annotation = method.getAnnotation(Test.class);

            if (annotation != null && annotation.enabled()) {
                System.out.println("Running test: " + annotation.description());
                method.invoke(instance);
            }
        }
    }
}
```

16. Java Design Patterns

Singleton Pattern:

```
// Eager initialization
public class EagerSingleton {
    private static final EagerSingleton INSTANCE = new EagerSingleton();

    private EagerSingleton() {}

    public static EagerSingleton getInstance() {
        return INSTANCE;
    }
}

// Lazy initialization (thread-safe)
public class LazySingleton {
    private static volatile LazySingleton instance;

    private LazySingleton() {}

    public static LazySingleton getInstance() {
        if (instance == null) {
            synchronized (LazySingleton.class) {
                if (instance == null) {
                    instance = new LazySingleton();
                }
            }
        }
        return instance;
    }
}

// Enum Singleton (Java 5+, preferred method)
public enum EnumSingleton {
    INSTANCE;

    public void doSomething() {
        // Singleton functionality
    }
}
```

Factory Pattern:


```
// Product interface
interface Vehicle {
    void drive();
}

// Concrete products
class Car implements Vehicle {
    @Override
    public void drive() {
        System.out.println("Driving a car");
    }
}

class Motorcycle implements Vehicle {
    @Override
    public void drive() {
        System.out.println("Riding a motorcycle");
    }
}

// Factory
class VehicleFactory {
    public Vehicle createVehicle(String type) {
        if ("car".equalsIgnoreCase(type)) {
            return new Car();
        } else if ("motorcycle".equalsIgnoreCase(type)) {
            return new Motorcycle();
        } else {
            throw new IllegalArgumentException("Unknown vehicle type");
        }
    }
}

// Usage
VehicleFactory factory = new VehicleFactory();
Vehicle car = factory.createVehicle("car");
car.drive(); // "Driving a car"
```

Builder Pattern:


```
class Person {  
    // Required parameters  
    private final String firstName;  
    private final String lastName;  
  
    // Optional parameters  
    private final int age;  
    private final String phone;  
    private final String address;  
  
    private Person(Builder builder) {  
        this.firstName = builder.firstName;  
        this.lastName = builder.lastName;  
        this.age = builder.age;  
        this.phone = builder.phone;  
        this.address = builder.address;  
    }  
  
    // Builder class  
    public static class Builder {  
        // Required parameters  
        private final String firstName;  
        private final String lastName;  
  
        // Optional parameters with default values  
        private int age = 0;  
        private String phone = "";  
        private String address = "";  
  
        public Builder(String firstName, String lastName) {  
            this.firstName = firstName;  
            this.lastName = lastName;  
        }  
  
        public Builder age(int age) {  
            this.age = age;  
            return this;  
        }  
  
        public Builder phone(String phone) {  
            this.phone = phone;  
            return this;  
        }  
  
        public Builder address(String address) {  
            this.address = address;  
        }  
    }  
}
```

```
        return this;
    }

    public Person build() {
        return new Person(this);
    }
}

// Usage
Person person = new Person.Builder("John", "Doe")
    .age(30)
    .phone("1234567890")
    .address("123 Main St")
    .build();
```

Observer Pattern:


```
import java.util.ArrayList;
import java.util.List;

// Observer interface
interface Observer {
    void update(String message);
}

// Concrete observer
class User implements Observer {
    private String name;

    public User(String name) {
        this.name = name;
    }

    @Override
    public void update(String message) {
        System.out.println(name + " received message: " + message);
    }
}

// Subject (Observable)
class NewsAgency {
    private List<Observer> observers = new ArrayList<>();
    private String news;

    public void addObserver(Observer observer) {
        observers.add(observer);
    }

    public void removeObserver(Observer observer) {
        observers.remove(observer);
    }

    public void setNews(String news) {
        this.news = news;
        notifyObservers();
    }

    private void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(news);
        }
    }
}
```

```
// Usage
NewsAgency agency = new NewsAgency();
Observer user1 = new User("User 1");
Observer user2 = new User("User 2");

agency.addObserver(user1);
agency.addObserver(user2);
agency.setNews("Breaking news!");
// Output:
// User 1 received message: Breaking news!
// User 2 received message: Breaking news!
```

Strategy Pattern:


```
// Strategy interface
interface PaymentStrategy {
    void pay(int amount);
}

// Concrete strategies
class CreditCardPayment implements PaymentStrategy {
    private String cardNumber;

    public CreditCardPayment(String cardNumber) {
        this.cardNumber = cardNumber;
    }

    @Override
    public void pay(int amount) {
        System.out.println(amount + " paid with credit card " + cardNumber);
    }
}

class PayPalPayment implements PaymentStrategy {
    private String email;

    public PayPalPayment(String email) {
        this.email = email;
    }

    @Override
    public void pay(int amount) {
        System.out.println(amount + " paid using PayPal account " + email);
    }
}

// Context
class ShoppingCart {
    private PaymentStrategy paymentStrategy;

    public void setPaymentStrategy(PaymentStrategy paymentStrategy) {
        this.paymentStrategy = paymentStrategy;
    }

    public void checkout(int amount) {
        paymentStrategy.pay(amount);
    }
}

// Usage
```

```
ShoppingCart cart = new ShoppingCart();
```

```
// Pay with credit card
```

```
cart.setPaymentStrategy(new CreditCardPayment("1234-5678-9012-3456"));
```

```
cart.checkout(100);
```

```
// Pay with PayPal
```

```
cart.setPaymentStrategy(new PayPalPayment("user@example.com"));
```

```
cart.checkout(200);
```

Decorator Pattern:


```
// Component interface
interface Coffee {
    String getDescription();
    double getCost();
}

// Concrete component
class SimpleCoffee implements Coffee {
    @Override
    public String getDescription() {
        return "Simple coffee";
    }

    @Override
    public double getCost() {
        return 2.0;
    }
}

// Decorator abstract class
abstract class CoffeeDecorator implements Coffee {
    protected Coffee decoratedCoffee;

    public CoffeeDecorator(Coffee coffee) {
        this.decoratedCoffee = coffee;
    }

    @Override
    public String getDescription() {
        return decoratedCoffee.getDescription();
    }

    @Override
    public double getCost() {
        return decoratedCoffee.getCost();
    }
}

// Concrete decorators
class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {

```

```

        return decoratedCoffee.getDescription() + ", milk";
    }

    @Override
    public double getCost() {
        return decoratedCoffee.getCost() + 0.5;
    }
}

class SugarDecorator extends CoffeeDecorator {
    public SugarDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return decoratedCoffee.getDescription() + ", sugar";
    }

    @Override
    public double getCost() {
        return decoratedCoffee.getCost() + 0.2;
    }
}

// Usage
Coffee coffee = new SimpleCoffee();
System.out.println(coffee.getDescription() + ": $" + coffee.getCost());

Coffee milkCoffee = new MilkDecorator(coffee);
System.out.println(milkCoffee.getDescription() + ": $" + milkCoffee.getCost());

Coffee sweetMilkCoffee = new SugarDecorator(milkCoffee);
System.out.println(sweetMilkCoffee.getDescription() + ": $" + sweetMilkCoffee.getCost());

```

17. Java Best Practices and Tips

Coding Conventions:

- Use camelCase for method and variable names: `calculateTotal()`, `userAge`
- Use PascalCase for class names: `CustomerService`, `BankAccount`
- Use ALL_CAPS for constants: `MAX_SIZE`, `PI`
- Use meaningful names that reflect purpose
- Keep methods small and focused on a single task

- Follow the principle of least surprise

Exception Handling Best Practices:

- Be specific with exceptions: catch specific exceptions rather than generic ones
- Don't catch exceptions you can't handle properly
- Don't swallow exceptions: at least log them
- Use finally blocks or try-with-resources for cleanup
- Create custom exceptions for domain-specific error conditions
- Include useful information in exception messages

Performance Tips:

- Use StringBuilder for string concatenation in loops
- Prefer primitive types over boxed types when possible
- Use appropriate collection types for specific use cases
- Avoid premature optimization
- Use lazy initialization for expensive resources
- Be cautious with recursion to avoid stack overflow
- Use streams for bulk operations on collections

Security Best Practices:

- Validate all input, especially from external sources
- Use prepared statements to prevent SQL injection
- Don't store sensitive information in plain text
- Use secure random number generators for security purposes
- Follow the principle of least privilege
- Keep dependencies updated to patch security vulnerabilities
- Be careful with serialization and deserialization

Testing Tips:

- Write unit tests for all public methods
- Use mocking frameworks for dependencies
- Test edge cases and error conditions
- Use test-driven development (TDD) when appropriate
- Separate test code from production code
- Write readable and maintainable tests

- Use continuous integration to run tests automatically

Tools and Libraries:

- Build tools: Maven, Gradle
- Testing: JUnit, TestNG, Mockito
- Logging: Log4j, SLF4J, Logback
- JSON: Jackson, Gson
- HTTP clients: OkHttp, Apache HttpClient
- Dependency injection: Spring, Guice
- Database access: Hibernate, jOOQ
- REST APIs: Spring MVC, Jersey
- Utilities: Apache Commons, Guava

18. New Features in Modern Java

Java 8 Features:

- Lambda expressions
- Stream API
- Optional class
- Default methods in interfaces
- Method references
- New Date and Time API (java.time)
- CompletableFuture for async programming
- Nashorn JavaScript engine

Java 9 Features:

- Module system (Project Jigsaw)
- JShell (REPL)
- Improved Process API
- Collection factory methods
- Private methods in interfaces
- Try-with-resources improvements

Java 10 Features:

- Local variable type inference (`var` keyword)

- Unmodifiable collections enhancements
- Parallel full GC for G1
- Application class-data sharing

Java 11 Features:

- HTTP Client API
- String methods: `isBlank()`, `lines()`, `strip()`
- Files methods: `readString()`, `writeString()`
- Running source files directly
- Lambda parameter var syntax

Java 12-13 Features:

- Switch expressions (preview)
- Text blocks (preview)
- Pattern matching for instanceof (preview)
- Records (preview)

Java 14-16 Features:

- Switch expressions (standard)
- Records (standard)
- Pattern matching for instanceof (standard)
- Helpful NullPointerExceptions
- Foreign memory access API
- Vector API (incubator)

Java 17 Features (LTS):

- Sealed classes
- Strong encapsulation of JDK internals
- Pattern matching for switch (preview)
- Foreign function & memory API (incubator)

Java 18-21 Features:

- Pattern matching for switch
- Virtual threads
- Record patterns

- String templates
- Structured concurrency
- Scoped values
- Foreign function & memory API improvements